

```
def print_board(board):
    for row in board:
        print(" ".join("Q" if cell == 1 else "." for cell in row))
```

```
step_counter = 0 # global step counter
```

```
def is_safe(board, row, col, n):
    # Check column above
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check upper-right diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, n)):
        if board[i][j] == 1:
            return False

    return True
```

```
def solve_queens(board, row, n):
    """
    Attempts to place queens from row..n-1.
    Prints every placement and backtrack step.
    Returns True when a complete solution is found.
    """

    global step_counter

    # If we placed queens for all rows -> solution found
    if row == n:
```

```
return True
```

```
# If row already has a queen (pre-placed), skip to next row
```

```
if 1 in board[row]:
```

```
    return solve_queens(board, row + 1, n)
```

```
for col in range(n):
```

```
    if is_safe(board, row, col, n):
```

```
        # Place queen
```

```
        board[row][col] = 1
```

```
        step_counter += 1
```

```
        print(f"\nStep {step_counter}: Placed queen at (row={row}, col={col})")
```

```
        print_board(board)
```

```
        # Recurse to next row
```

```
        if solve_queens(board, row + 1, n):
```

```
            return True
```

```
    # Backtrack
```

```
    board[row][col] = 0
```

```
    step_counter += 1
```

```
    print(f"\nStep {step_counter}: Backtracked from (row={row}, col={col})")
```

```
    print_board(board)
```

```
# No valid position in this row
```

```
return False
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        n = int(input("Enter board size n (e.g., 8): ").strip())
```

```
        if n <= 0:
```

```

print("Board size must be positive.")

raise SystemExit

board = [[0 for _ in range(n)] for _ in range(n)]

place_initial = input("Do you want to place an initial queen? (y/n): ").strip().lower()

if place_initial == 'y':
    r = int(input(f"Enter initial queen row (0..{n-1}): ").strip())
    c = int(input(f"Enter initial queen column (0..{n-1}): ").strip())
    if not (0 <= r < n and 0 <= c < n):
        print("Invalid initial position. Exiting.")
        raise SystemExit
    board[r][c] = 1
    print(f"\nInitial queen placed at (row={r}, col={c}):")
    print_board(board)
    # Start solving from row 0 (rows with pre-placed queen are skipped)
    start_row = 0
else:
    start_row = 0

print("\nStarting step-wise solving...\n")
found = solve_queens(board, start_row, n)

if found:
    print("\n=== Final Solution ===\n")
    print_board(board)
    print(f"\nTotal steps (placements + backtracks): {step_counter}")
else:
    print("\nNo solution found from the given starting position.")
    print(f"Total steps attempted: {step_counter}")

```

```
except ValueError:
```

```
    print("Invalid input. Please enter integer values.")
```

OUTPUT :

Enter board size n (e.g., 8): 8

Do you want to place an initial queen? (y/n): y

Enter initial queen row (0..7): 2

Enter initial queen column (0..7): 5